

数据集的准备

这次案例我们需要完成对YOLO模型的训练使用的模型为YOLOV5-Lite

首先来介绍目标检测任务的基本概念：

目标检测相关概念

1. 目标检测 (Object Detection) :
 - 任务：不仅识别图像中的对象，还确定它们的位置。
 - 输出：通常是一组边界框 (bounding boxes) 和相关的类别标签。
2. 边界框 (Bounding Box) :
 - 定义：一个矩形框，用来表示图像中目标的位置。
 - 参数：通常包括边界框的 x、y 坐标 (表示框的左上角位置) 以及宽度和高度。
3. 锚框 (Anchor Boxes) :
 - 定义：预定义的边界框形状，用于帮助模型定位目标。
 - 用途：YOLO 使用锚框来简化目标检测任务，通过预测相对于锚框的偏移量而不是直接预测边界框坐标。
4. 置信度 (Confidence Score) :
 - 定义：模型对其检测到的每个边界框包含一个目标的置信度。
 - 用途：用于区分目标和背景，以及在非极大值抑制 (NMS) 中筛选最终的检测结果。
5. 非极大值抑制 (Non-maximum Suppression, NMS) :
 - 定义：一种算法，用于筛选一组重叠的边界框，只保留最有可能包含目标的那些。
 - 用途：减少重复检测，确保每个目标只有一个边界框与之对应。
6. 类别概率：
 - 定义：模型预测每个边界框内目标属于不同类别的概率。
 - 用途：帮助确定边界框内的对象具体属于哪个类别。

准备数据集

首先我们需要用小车的摄像头拍一定数量的图片用于训练模型

5G小车摄像头采集图片程序

```
import cv2
import os

# 创建保存帧的文件夹
output_folder = '/home/pi/image/'
if not os.path.exists(output_folder):
    os.makedirs(output_folder)

# 打开摄像头（参数2表示第二个摄像头，可以改为视频文件路径）
cap = cv2.VideoCapture(2)
```

```

# 检查摄像头是否打开成功
if not cap.isOpened():
    print("Error: Could not open video source.")
    exit()

# 设置视频帧的宽度和高度
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

# 初始化帧计数器
frame_count = 1

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # 检查文件是否已经存在, 如果存在则增加frame_count
    frame_filename = os.path.join(output_folder, f'frame_{frame_count:04d}.png')
    while os.path.exists(frame_filename):
        frame_count += 1
        frame_filename = os.path.join(output_folder,
f'frame_{frame_count:04d}.png')

    # 保存当前帧为图片
    cv2.imwrite(frame_filename, frame)

    # 显示帧
    cv2.imshow('video', frame)

    # 按 'q' 键退出
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

    # 增加帧计数器
    frame_count += 1

# 释放资源
cap.release()
cv2.destroyAllWindows()

```

示例照片:



我们每个类型的照片都拍一定数量之后,打包压缩成.zip文件

数据集标注

打开浏览器 地址栏输入: <https://console.bce.baidu.com/easydata/app/>

登录进入如下页面:

The screenshot displays the Baidu EasyData console interface. The top navigation bar includes the Baidu logo, '百度智能云', and various service links. The left sidebar lists navigation options such as '数据集管理', '数据标注', '数据清洗', '数据增强', '数据回流', '数据查看', and '摄像头采集'. The main content area is titled '数据集管理' and provides an overview of the platform's capabilities. It includes a '产品介绍' (Product Introduction) section and a '我的数据集' (My Datasets) table. The table lists datasets with columns for '版本' (Version), '数据集ID', '数据集量', '最近导入状态', '标注类型 > 模板', '标注状态', '清洗状态', and '操作'. A dataset named 'V1' is shown with a status of '已完成' (Completed) and a label count of 1405/1405. The interface also features a search bar and various filters for dataset management.

百度智能云 控制台首页 华北 - 北京 请输入您想要搜索的产品、文档

EasyData数据...

default

数据管理

数据集

数据源

数据标注

标签组管理

在线标注

智能标注

多人标注

寻求标注支持

数据处理

数据清洗

数据增强

数据回流

回流配置

数据查看

摄像头采集

采集配置

数据集管理

平台支持统一纳管自训练模型的数据集，并支持自主版本迭代、数据查看、导入导出和删除等操作。

产品介绍

EasyData为百度大旗推出的一站式数据处理和服务平台，主要围绕AI模型开发过程中所需的数据采集、数据质检、数据智能处理、数据标注等环节提供完整的数据服务。目前EasyData已支持图片、文本、音频、视频、表格五类基础数据的处理，同时EasyData已与EasyDL、BML平台数据管理模块打通，EasyData处理的数据可直接应用于EasyDL、BML平台进行模型训练。

数据集

数据质检

数据智能处理

数据标注

支持采集图片类数据，可以从本地接入上传除敏感图片或通过接入云服务商调用数据接入图片

支持对图像数据进行质检，质检报告中的指标可作为数据（标注、清洗）的重要参考

支持对图片和文本数据进行清洗，以及对图片数据进行增强处理，您可按需选择数据智能处理功能

支持图片、文本、音频、视频数据标注，并支持丰富标注模板，个人在线标注及智能标注等多种方式

接入摄像头采集 云服务数据回流

数据清洗 数据增强

在线标注 智能标注 多人标注

+ 创建数据集

我的数据集

全部

输入数据集名称或ID

ABCEDEFG 数据集ID: 820167

新增版本 全部版本 删除

版本	数据集ID	数据量	最近导入状态	标注类型 > 模板	标注状态	清洗状态	操作
V1	2140879	1405	已完成	物体检测 > 自定义四边形标注	100% (1405/1405)	-	查看 导入 导出 标注 ...

AB 数据集ID: 818355

新增版本 全部版本 删除

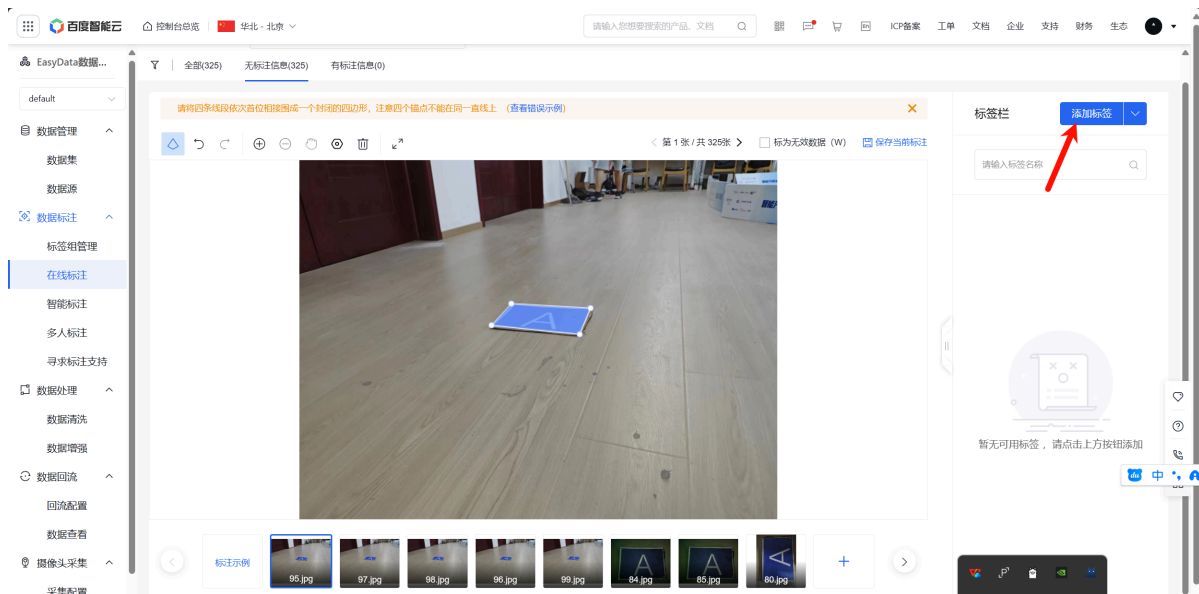
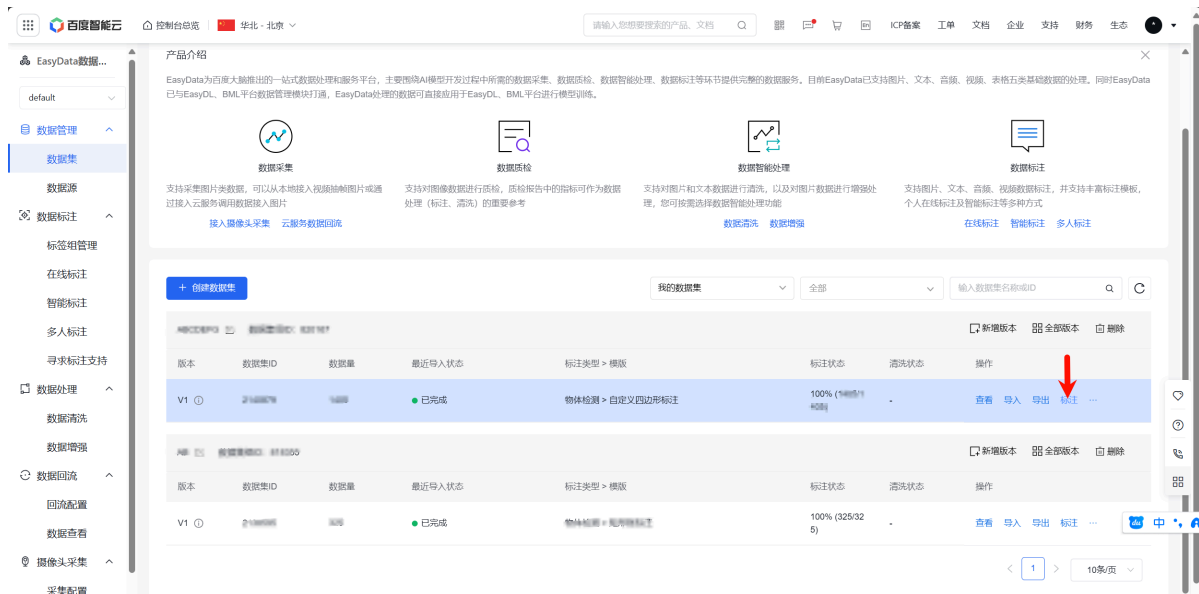
版本	数据集ID	数据量	最近导入状态	标注类型 > 模板	标注状态	清洗状态	操作
----	-------	-----	--------	-----------	------	------	----

The screenshot shows the 'EasyData数据管理' (EasyData Data Management) console. The left sidebar contains a navigation menu with items like '数据集' (Dataset), '数据源' (Data Source), '数据标注' (Data Annotation), '数据标注管理' (Data Annotation Management), '在线标注' (Online Annotation), '智能标注' (Smart Annotation), '多人标注' (Multi-person Annotation), '寻求标注支持' (Seek Annotation Support), '数据处理' (Data Processing), '数据清洗' (Data Cleaning), '数据增强' (Data Augmentation), '数据回流' (Data Backflow), '回流配置' (Backflow Configuration), '数据查看' (Data Viewing), '摄像头采集' (Camera Collection), and '采集配置' (Collection Configuration). The main content area is titled '基本信息' (Basic Information) and includes the following details:

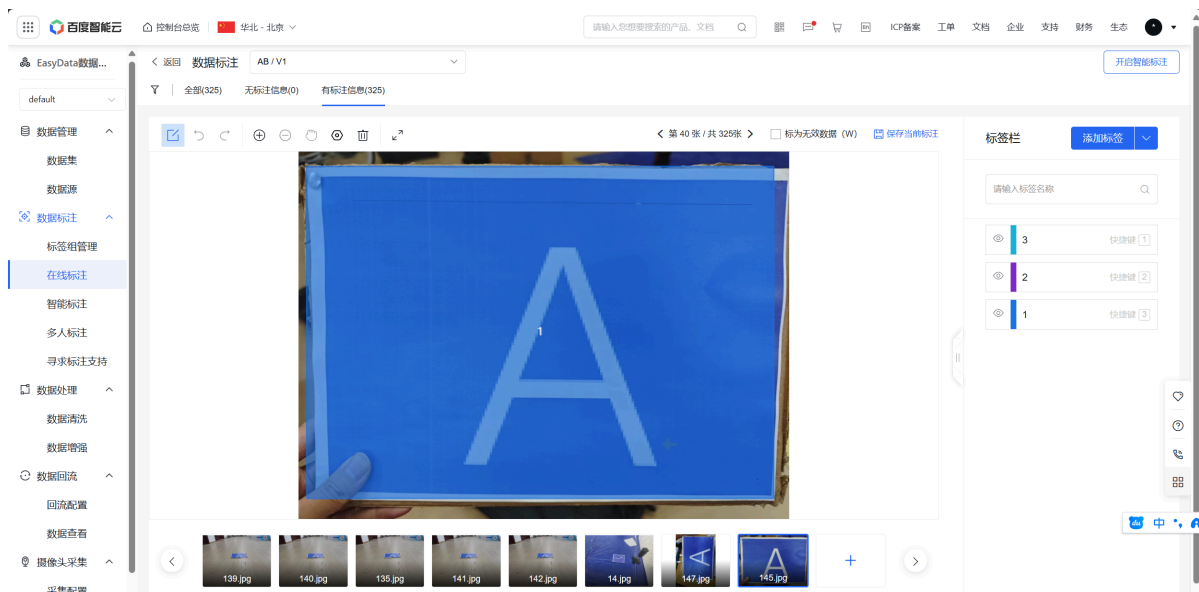
- 数据集名称:** 我的数据集 (Dataset Name: My Dataset)
- 数据类型:** 图片 (Image) (Selected), 文本 (Text), 音频 (Audio), 视频 (Video), 表格 (Table), 跨模态 (Cross-modal)
- 数据集版本:** V1
- 标注类型:** 图像分类 (Image Classification), 物体检测 (Object Detection) (Selected), 图像分割 (Image Segmentation), OCR标注 (OCR Annotation), 点标注 (Point Annotation), 线标注 (Line Annotation), 3D框标注 (3D Box Annotation), 混合标注 (Mixed Annotation)
- 标注模板:** ☐ 矩形框标注 (Rectangular Box Annotation), ☒ 自定义四边形标注 (Custom Quadrilateral Annotation)
- 保存位置:** ☒ 平台存储 (Platform Storage), ☐ BOS存储 (BOS Storage)

At the bottom of the page, there are three buttons: '创建并导入' (Create and Import), '完成创建' (Complete Creation), and '取消' (Cancel). A red arrow points to the '创建并导入' button.

上传图片打包好的压缩包,回到原始界面开始标注



注意 标注时尽量标注准确! 选择好每个类别对应的标签,没有目标或者目标看不清的为无效图像



标注完成后,选择导出 导出为平台默认格式!

数据集格式转换

导出完毕后,我们需要整理我们的数据集的文件结构为:

```
train: ../coco/images/train2017/
val: ../coco/images/val2017/
├─ images                # xx.jpg example
│   └─ train2017
│       ├── 000001.jpg
│       ├── 000002.jpg
│       └── 000003.jpg
│   └─ val2017
│       ├── 100001.jpg
│       ├── 100002.jpg
│       └── 100003.jpg
└─ labels                # xx.txt example
    ├── train2017
    │   ├── 000001.txt
    │   ├── 000002.txt
    │   └── 000003.txt
    └─ val2017
        ├── 100001.txt
        ├── 100002.txt
        └── 100003.txt
```

这里附上转换数据集格式脚本,因为YOLO模型需要的格式为 标签 物体标注框的四个点

```
import json
import os
from PIL import Image

def convert_json_to_txt(json_path, txt_path, image_path):
    with open(json_path, 'r') as json_file:
        data = json.load(json_file)

    labels = data.get("labels", [])

    if not labels:
        return

    # Open the image to get its dimensions
    with Image.open(image_path) as img:
        image_width, image_height = img.size

    with open(txt_path, 'w') as txt_file:
        for label in labels:
            name = label.get("name")
            x1, y1, x2, y2 = label.get("x1"), label.get("y1"), label.get("x2"),
            label.get("y2")

            width = x2 - x1
            height = y2 - y1
```

```

        center_x = x1 + width / 2
        center_y = y1 + height / 2

        # Normalize the coordinates using the actual image dimensions
        normalized_center_x = center_x / image_width
        normalized_center_y = center_y / image_height
        normalized_width = width / image_width
        normalized_height = height / image_height

        txt_file.write(
            f"{name} {normalized_center_x} {normalized_center_y}
{normalized_width} {normalized_height}\n")

# Example usage
json_folder = "你的数据集文件地址"
image_folder = "你的数据集文件地址"
txt_folder = "你的数据集文件地址/labels"

if not os.path.exists(txt_folder):
    os.makedirs(txt_folder)

for json_file_name in os.listdir(json_folder):
    if json_file_name.endswith('.json'):
        json_path = os.path.join(json_folder, json_file_name)
        txt_file_name = json_file_name.replace('.json', '.txt')
        txt_path = os.path.join(txt_folder, txt_file_name)

        image_file_name = json_file_name.replace('.json', '.png')
        image_path = os.path.join(image_folder, image_file_name)

        if os.path.exists(image_path):
            convert_json_to_txt(json_path, txt_path, image_path)
        else:
            print(f"Image file {image_file_name} not found, skipping...")

```

将程序里面 你的数据集文件地址替换为你导出出来的文件地址

然后进行训练集和测试集的划分:

```

import os
import shutil
import random

def split_dataset(image_folder, label_folder, output_folder, train_ratio=0.8):
    # Create directories for train and val sets
    train_image_dir = os.path.join(output_folder, 'images', 'train2017')
    val_image_dir = os.path.join(output_folder, 'images', 'val2017')
    train_label_dir = os.path.join(output_folder, 'labels', 'train2017')
    val_label_dir = os.path.join(output_folder, 'labels', 'val2017')

    os.makedirs(train_image_dir, exist_ok=True)

```



```

os.makedirs(val_image_dir, exist_ok=True)
os.makedirs(train_label_dir, exist_ok=True)
os.makedirs(val_label_dir, exist_ok=True)

# Get list of image files and corresponding label files
image_files = [f for f in os.listdir(image_folder) if f.endswith('.png')]
random.shuffle(image_files)

train_size = int(len(image_files) * train_ratio)
train_files = image_files[:train_size]
val_files = image_files[train_size:]

def copy_files(file_list, source_image_folder, source_label_folder,
dest_image_folder, dest_label_folder, start_index):
    for index, image_file in enumerate(file_list):
        base_name = f"{start_index + index:06d}"
        src_image_path = os.path.join(source_image_folder, image_file)
        dest_image_path = os.path.join(dest_image_folder, f"{base_name}.png")
        shutil.copy(src_image_path, dest_image_path)

        label_file = image_file.replace('.png', '.txt')
        src_label_path = os.path.join(source_label_folder, label_file)
        dest_label_path = os.path.join(dest_label_folder, f"{base_name}.txt")
        if os.path.exists(src_label_path):
            shutil.copy(src_label_path, dest_label_path)

# Copy training files
copy_files(train_files, image_folder, label_folder, train_image_dir,
train_label_dir, start_index=1)

# Copy validation files
copy_files(val_files, image_folder, label_folder, val_image_dir,
val_label_dir, start_index=100001)

# Example usage
image_folder = '你的数据集文件地址/images'
label_folder = '你的数据集文件地址/labels'
output_folder = '你的数据集文件地址'

split_dataset(image_folder, label_folder, output_folder)

```

运行完毕后,请仔细检查文件系统格式是否正确!